

Project Title

AI Based Assignment Checker

By

Shanza Habib & Hina Shafiq

Roll No: 123355 & 123360

Session: 2022-2026

Supervisor: Sir Ghulam Ghos



**Bachelor of Science in Computer Science, GC University
Faisalabad**

January 2026

Certificate of Approval

This is to certify that the project titled "**AI Based Assignment Checker**" submitted by **SHANZA HABIB & HINA SHAFIQ (Roll No: 123355 & 123360)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology has been examined and approved.

Name: Sir. Ghulam Ghos

Name: Sir Jameel Malik

Signature: _____

Signature: _____

Date: _____

Date: _____

Declaration

I hereby declare that this project titled "**AI Based Assignment Checker**" is my own work and has not been submitted elsewhere for any other degree or qualification

Name: SHANZA HABIB

Roll No: 123355

Signature: _____

Date: _____

Name: HINA SHAFIQ

Roll No: 123360

Signature: _____

Date: _____

Acknowledgements

All praise, thanks, glorification, and gratitude are for **ALLAH** Almighty. I achieved my goal by **ALLAH's** infinite mercy and grace. I say Alhamdulillah for HIS uncountable blessings. After **ALLAH**, thanks to last Prophet **Muhammad** (ﷺ) who guided us toward the right path. Beloved prophet (ﷺ) says “Education is equally important for men and women”.

Very special and humble thanks to my father. He is my main supporter to continue the studies. My father instilled iron determination in me. I just want to say, he is great, he is genius. I also express special thanks to my mother. She always prayed for me and improves my confidence to continue the higher study. My mother has always been very caring about my health, studies, and career. I would like to pay my heartiest gratitude to my sister and brothers for being a great source of support and encouragement to complete this work. Especially, thanks to my brothers who guided and helped me all along the way.

I would like to express my sincere gratitude to my supervisor **Mr. Ghulam Ghos** for the continuous support of my research work, for their patience, motivation, enthusiasm, and immense knowledge. Their guidance helped me during all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my research work.

In addition, I would like to acknowledge important role of **sir Jameel**, whose help and assistance remained a source of motivation for me throughout this task.

SHANZA HABIB
123355

HINA SHAFIQ
123360

Abstract

This paper presents an AI Based Assignment Checker...

TABLE OF CONTENTS

Chapter 1: Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Motivation	1
1.4 Aims and Objectives	1
1.5 Project Scope	2
1.6 Document Structure	2
Chapter 2: Literature Review & Background Study	3
2.1 Introduction to the Domain	3
2.2 Existing Systems Analysis	3
2.3 Comparative Analysis	3
2.4 Identified Research Gaps	4
2.5 Proposed Solution Overview	4
Chapter 3: Requirement Analysis & Specification	5
3.1 Stakeholder Analysis	5
3.2 Functional Requirements	5
3.3 Non-Functional Requirements	6
3.4 Use Case Modeling	7
3.5 User Stories	9
3.6 Hardware and Software Requirements	10
Chapter 4: System Design	11
4.1 Design Principles and Methodology	11
4.2 System Architecture	11
4.3 Data Storage Design	12
4.4 API Design	13
4.5 Process Design	14
4.6 Security Design	16
4.7 UI/UX Design	16
Chapter 5: Implementation & Development	18
5.1 Development Environment Setup	18

5.2 Backend Module Implementation	18
5.3 Frontend Module Implementation	20
5.4 Key Code Explanations	21
5.5 Challenges Faced and Solutions	21
Chapter 6: Testing & Validation	23
6.1 Testing Strategy	23
6.2 Functional Test Cases	23
6.3 Edge Case Testing	25
6.4 Testing Summary	25
Chapter 7: Results & Discussion	27
7.1 System Implementation Results	27
7.2 Achievement of Objectives	29
7.3 Performance Evaluation	30
7.4 Discussion	31
7.5 Limitations Encountered	31
Chapter 8: Conclusion & Future Work	32
8.1 Conclusion	32
8.2 Limitations	32
8.3 Future Enhancements	32
8.4 Social Impact and Applications	33
References	34
Appendix	35
Appendix A - Installation and Setup Guide	35
Appendix B - API Endpoint Reference	35
Appendix C - Storage Schema Reference	35

List of FIGURES

Figure 3.1 - System Level Use Case Diagram.....	8
Figure 3.2 - Student Use Case Diagram.....	8
Figure 3.3 - Teacher Use Case Diagram.....	9
Figure 4.1 - High Level System Architecture Diagram	11
Figure 4.2 - Three-Tier Architecture Diagram	12
Figure 4.3 - Assignment Submission Sequence Diagram	15
Figure 4.4 - AI Checking Pipeline Sequence Diagram	15
Figure 4.5 - Student Dashboard UI Design.....	16
Figure 4.6 - Teacher Dashboard UI Design	17
Figure 5.1 - Project Directory Structure	18
Figure 7.1 - Student Dashboard with Assignment List	27
Figure 7.2 - Assignment Submission Page with Drag and Drop	27
Figure 7.3 - Result Page with Score Breakdown and Grammar Errors.....	28
Figure 7.4 - Teacher Dashboard with Analytics Charts	28
Figure 7.5 - Plagiarism Detection Result Showing Matched Assignment	29
Figure 7.6 - Teacher Grade Override Panel	29

Chapter 1: Introduction

1.1 Project Overview

The AI Assignment Checker is an automated grading system designed for educational institutions. It allows students to submit their assignments in PDF, DOCX, or TXT format and receive instant AI-powered feedback. Teachers can review all submissions, override grades, and monitor class-wide performance through a dedicated analytics dashboard.

The system integrates three independent evaluation modules: a grammar checker powered by LanguageTool, a plagiarism detector built using TF-IDF cosine similarity, and an AI content quality evaluator using the Llama 2 language model. Results are combined using a weighted formula to produce a final score and letter grade.

1.2 Problem Statement

Manual grading of student assignments is a time-consuming and inconsistent process. Teachers in large classrooms often struggle to provide timely, detailed, and objective feedback. The lack of automated tools for detecting plagiarism and evaluating content quality leads to unfair assessments and discourages academic integrity.

Existing commercial grading platforms are either expensive, cloud-dependent, or lack transparency in their scoring methodology. There is a clear need for an open, locally-hosted solution that provides detailed, explainable feedback across multiple dimensions of assignment quality.

1.3 Project Motivation

This project was motivated by the growing demand for AI-assisted educational tools in Pakistani universities, where class sizes are large and grading resources are limited. By providing a free, self-hosted platform, the system empowers both students and teachers to engage with detailed performance metrics without depending on expensive third-party services.

The use of locally-run AI (Llama 2 via Ollama) and a self-hosted grammar tool (LanguageTool via Docker) ensures data privacy and eliminates recurring subscription costs.

1.4 Aims and Objectives

The primary aim of this project is to develop a fully functional, AI-powered assignment grading platform. The specific objectives are:

- To implement a multi-format file upload and text extraction pipeline supporting PDF, DOCX, and TXT files.
- To integrate a self-hosted grammar checking service using LanguageTool and compute a grammar score.
- To develop a custom plagiarism detection module using TF-IDF cosine similarity against all previously submitted assignments.
- To use the Llama 2 language model to evaluate content quality across four dimensions: relevance, structure, depth, and clarity.
- To implement a weighted final scoring formula and automatic grade assignment.
- To build a role-based web application with separate student and teacher interfaces.
- To provide teachers with the ability to override grades and add comments visible to students.

1.5 Project Scope

The project scope encompasses the following components:

- A FastAPI-based REST API backend with MongoDB for persistent storage.
- A React 19 + Vite frontend with role-based routing for students and teachers.
- Three independent AI/ML checking services running locally.
- A student portal for assignment submission and result viewing.
- A teacher portal for class management, grading, and analytics.

The system is designed for single-institution deployment and does not include multi-tenancy, email notifications, or integration with external LMS platforms in its current scope.

1.6 Document Structure

This document is organized into eight chapters as follows:

- Chapter 2 reviews existing automated grading systems and identifies research gaps.
- Chapter 3 details the functional and non-functional requirements of the system.
- Chapter 4 describes the system design including architecture, database schema, and UI design.
- Chapter 5 covers the implementation details of all backend and frontend modules.
- Chapter 6 presents the testing strategy and test case results.
- Chapter 7 discusses the results and evaluates the system against its objectives.
- Chapter 8 concludes the report and outlines future enhancement directions.

Chapter 2: Literature Review & Background Study

2.1 Introduction to the Domain

Automated essay and assignment scoring (AES) has been an active research area since the 1960s. Early systems like Project Essay Grade (PEG) relied on surface-level text features such as word count, sentence length, and punctuation frequency. Modern systems leverage natural language processing (NLP) and machine learning to evaluate content quality, grammatical correctness, and semantic relevance.

The emergence of large language models (LLMs) such as GPT-4, LLaMA, and Gemini has opened new possibilities for nuanced content evaluation, enabling systems to assess higher-order writing qualities like argumentation depth and logical clarity.

2.2 Existing Systems Analysis

Several automated grading platforms are currently in use in educational settings. The most prominent ones include:

- Turnitin: Widely used for plagiarism detection. It maintains a large proprietary database but requires expensive institutional subscriptions.
- Grammarly: A grammar and style correction tool. It does not provide academic scoring or plagiarism detection against peer submissions.
- Gradescope: An AI-assisted grading tool for handwritten and typed submissions, primarily for STEM subjects.
- ETS e-rater: A machine learning-based scoring engine used in standardized testing. Not available for general educational use.

2.3 Comparative Analysis

Table 2.1 provides a feature-by-feature comparison of existing systems against the proposed solution.

Table 2.1 - Comparative Analysis of Existing Systems

Feature	Turnitin	Grammarly	Gradescope	AI Assignment Checker
Plagiarism Detection	Yes	No	No	Yes (Custom TF-IDF)
Grammar Checking	No	Yes	No	Yes (LanguageTool)

Feature	Turnitin	Grammarly	Gradescope	AI Assignment Checker
AI Content Evaluation	No	Partial	No	Yes (Llama 2)
Local / Self-hosted	No	No	No	Yes
Free / Open Source	No	Freemium	No	Yes
Role-based Access	Yes	No	Yes	Yes
Teacher Grade Override	No	No	Yes	Yes
Analytics Dashboard	Limited	No	Yes	Yes

2.4 Identified Research Gaps

The analysis of existing systems reveals the following key gaps:

- No existing free, self-hosted tool combines grammar checking, plagiarism detection, and AI content scoring in a single platform.
- Most tools are cloud-based, raising data privacy concerns for institutions in developing countries.
- Teacher-side tools for grade override, commenting, and per-student analytics are absent in most solutions.
- Existing plagiarism detectors compare against internet content but not against the institution's own submission history.

2.5 Proposed Solution Overview

The AI Assignment Checker addresses these gaps by providing an integrated, locally-deployable platform. It uses three lightweight, open-source tools that can run on commodity hardware: LanguageTool for grammar, scikit-learn TF-IDF for plagiarism, and Llama 2 via Ollama for content evaluation. The system is built with FastAPI and React, ensuring a modern, responsive user experience for both students and teachers.

Chapter 3: Requirement Analysis & Specification

3.1 Stakeholder Analysis

The primary stakeholders of the AI Assignment Checker are:

- Students: Submit assignments, view detailed results, track progress over time.
- Teachers: Review all submissions, override grades, add comments, view analytics.
- System Administrator: Manages server deployment, MongoDB, and external service configuration.

3.2 Functional Requirements

Table 3.1 lists the core functional requirements of the system.

Table 3.1 - Functional Requirements

ID	Module	Requirement Description	Priority
FR-01	Authentication	System shall support user registration with name, email, password, and role selection (student/teacher).	High
FR-02	Authentication	System shall authenticate users via email and password and return a JWT token.	High
FR-03	File Upload	System shall accept PDF, DOCX, and TXT files up to 5 MB.	High
FR-04	File Upload	System shall extract plain text from uploaded files.	High
FR-05	Grammar Check	System shall check grammar using LanguageTool and return error details and a score out of 10.	High
FR-06	Plagiarism	System shall compare submission text against all existing submissions using TF-IDF cosine similarity.	High
FR-07	Plagiarism	System shall flag assignments with similarity score $\geq 70\%$ as plagiarized.	High

ID	Module	Requirement Description	Priority
FR-08	Content AI	System shall send assignment text to Llama 2 and receive scores for relevance, structure, depth, and clarity.	High
FR-09	Scoring	System shall compute final score using Grammar (30%), Content (50%), Plagiarism (20%) weights.	High
FR-10	Scoring	System shall override final score to 0 if plagiarism is detected.	High
FR-11	Student Portal	Students shall view a list of their submissions with status and scores.	High
FR-12	Student Portal	Students shall view a detailed result page for each graded submission.	High
FR-13	Student Portal	Students shall view a progress dashboard with score trends and charts.	Medium
FR-14	Teacher Portal	Teachers shall view all student submissions with filtering and sorting options.	High
FR-15	Teacher Portal	Teachers shall override the AI grade and add comments on any submission.	High
FR-16	Teacher Portal	Teachers shall access an analytics dashboard showing class-wide statistics.	Medium

3.3 Non-Functional Requirements

Table 3.2 specifies the non-functional requirements of the system.

Table 3.2 - Non-Functional Requirements

Category	Requirement
Performance	Grammar and plagiarism checks should complete within 10 seconds. Content AI check may take up to 120 seconds depending on Llama 2 availability.

Category	Requirement
Reliability	The system shall return neutral fallback scores if LanguageTool or Ollama is unavailable, ensuring the grading pipeline never crashes.
Security	All API endpoints except login and register shall require a valid JWT token. Passwords shall be stored using bcrypt hashing.
Usability	The UI shall be fully responsive and usable on mobile, tablet, and desktop devices.
Scalability	The system shall support concurrent submissions using FastAPI's asynchronous background task processing.
Privacy	All data shall be stored locally. No student data shall be transmitted to external servers.
Maintainability	The codebase shall be modular with separate services for grammar, plagiarism, and content evaluation.

3.4 Use Case Modeling

The system includes three primary use case groups: overall system-level interactions, student-specific actions, and teacher-specific actions.

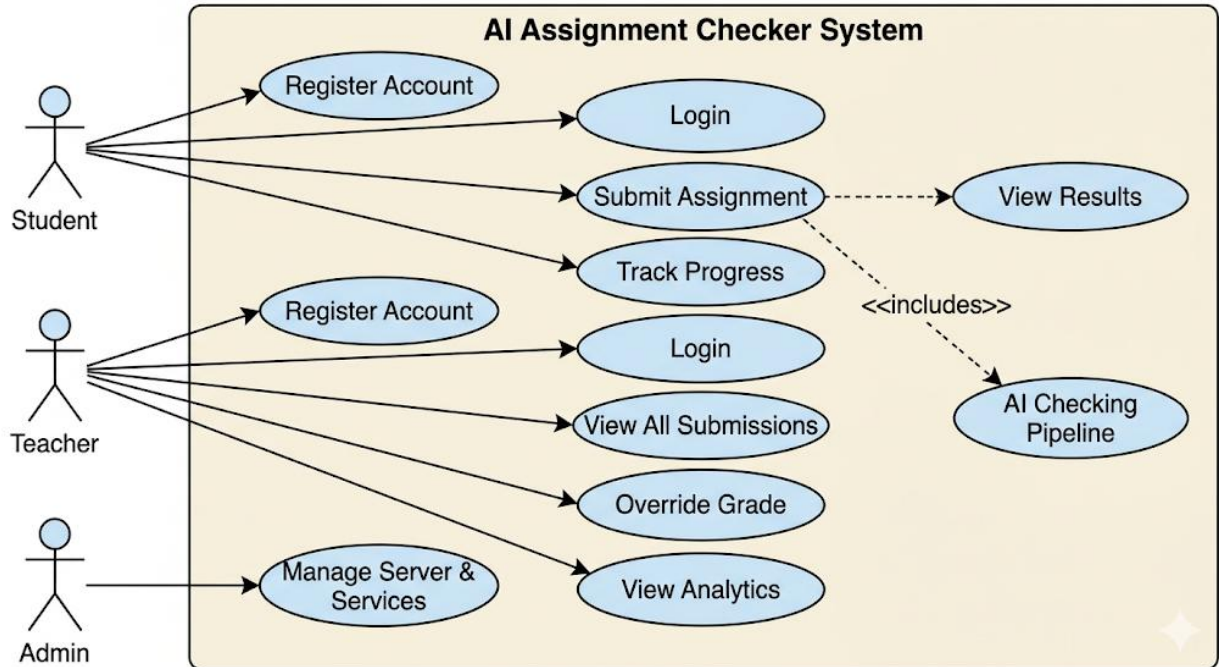


Figure 3.1 - System Level Use Case Diagram

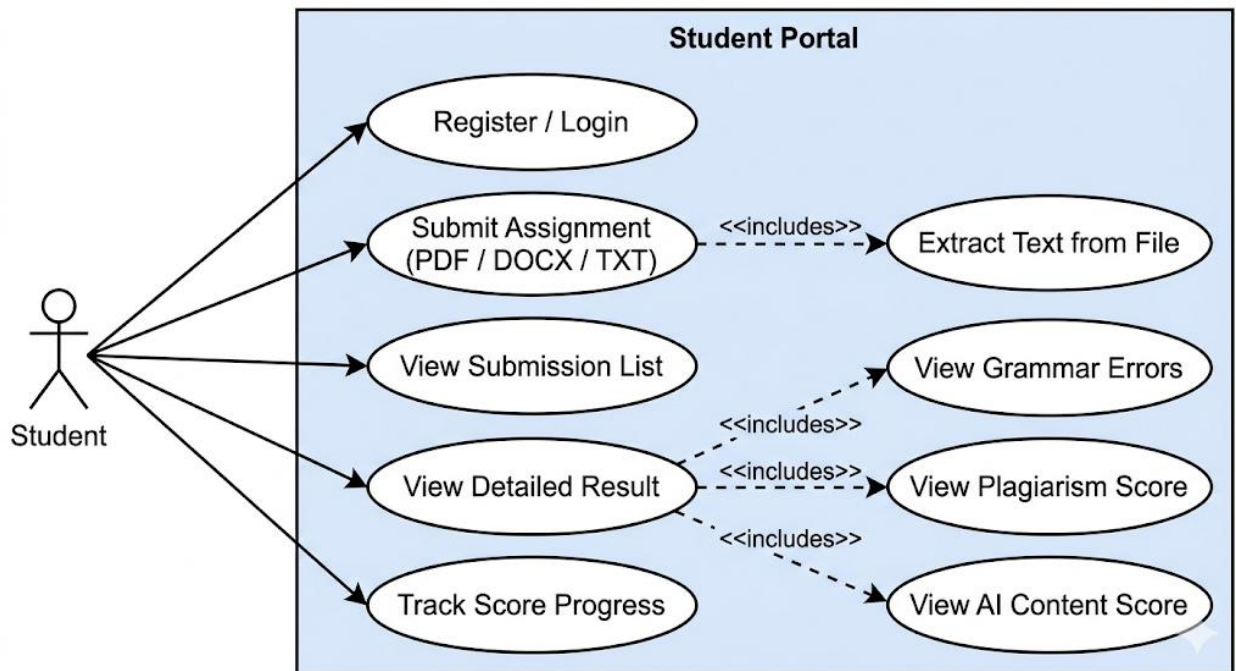


Figure 3.2 - Student Use Case Diagram

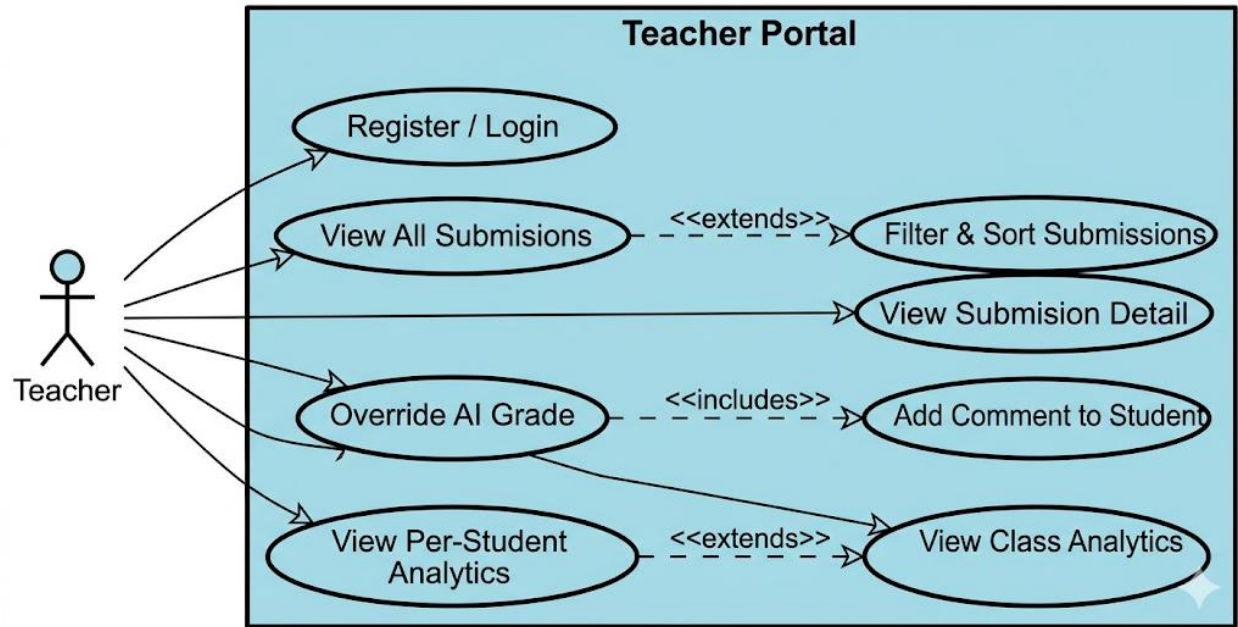


Figure 3.3 - Teacher Use Case Diagram

3.5 User Stories

Table 3.3 presents the key user stories for the system.

Table 3.3 - User Stories

ID	Role	User Story
US-01	Student	As a student, I want to upload my assignment file so that I can receive AI-powered feedback on my work.
US-02	Student	As a student, I want to see my grammar errors with suggestions so that I can improve my writing.
US-03	Student	As a student, I want to know if my work is flagged for plagiarism so that I can understand my originality score.
US-04	Student	As a student, I want to view my score trends over time so that I can track my academic progress.
US-05	Teacher	As a teacher, I want to view all submissions in one place so that I can efficiently manage my class.

ID	Role	User Story
US-06	Teacher	As a teacher, I want to override the AI grade so that I can apply my professional judgment.
US-07	Teacher	As a teacher, I want to see analytics for each student so that I can identify students who need extra support.

3.6 Hardware and Software Requirements

Table 3.4 lists the minimum hardware and software requirements for deploying the system.

Table 3.4 - Hardware and Software Requirements

Component	Minimum Requirement
Processor	Intel Core i5 (8th gen) or equivalent, 4 cores
RAM	8 GB (16 GB recommended for running Llama 2)
Storage	20 GB free disk space for model weights and database
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 12+
Python	Python 3.10 or higher
Node.js	Node.js 20 or higher
MongoDB	MongoDB 6.0 or higher (local instance)
Docker	Docker Desktop (for LanguageTool container)
Ollama	Ollama CLI with llama2 model pulled

Chapter 4: System Design

4.1 Design Principles and Methodology

The system was designed following a layered, service-oriented architecture. The key design principles applied are:

- Separation of Concerns: Each AI checking service (grammar, plagiarism, content) is implemented as an independent module.
- Asynchronous Processing: Assignment checks run as background tasks so the API remains responsive.
- Graceful Degradation: If any external service (LanguageTool, Ollama) is unavailable, neutral fallback scores are returned.
- Role-Based Access Control: All API routes are protected by JWT authentication with role-specific middleware.

4.2 System Architecture

The system follows a three-tier architecture: Presentation Layer (React frontend), Business Logic Layer (FastAPI backend), and Data Layer (MongoDB). The three AI services communicate with the backend through internal async function calls rather than separate microservice endpoints.

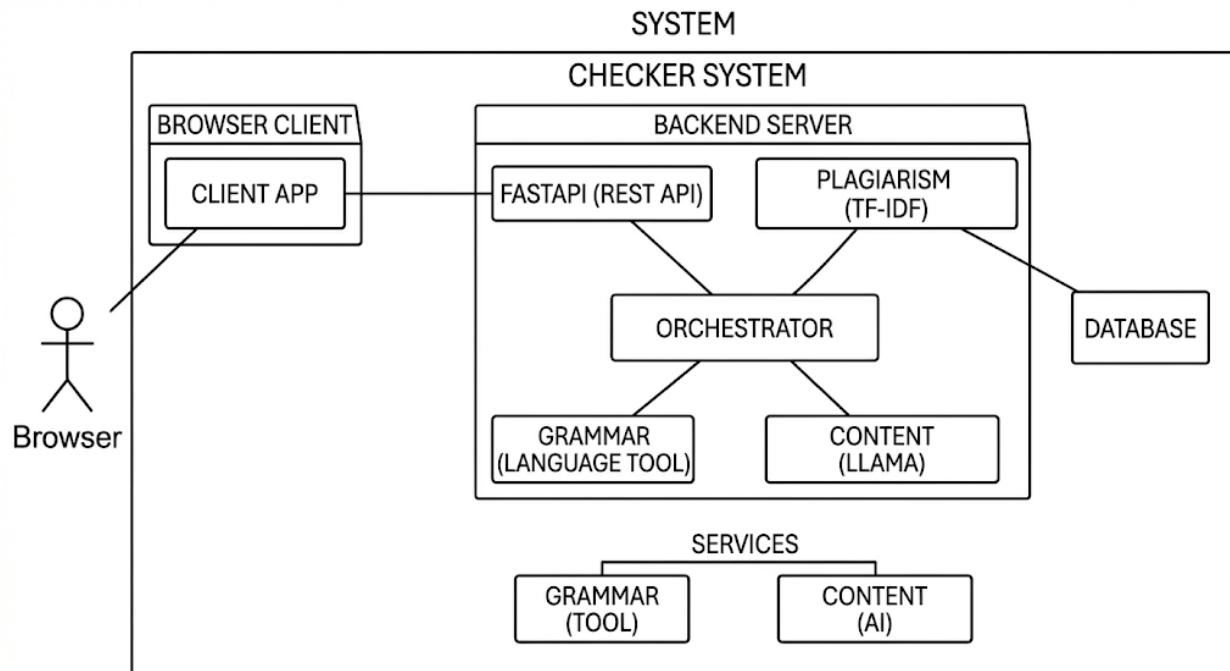


Figure 4.1 - High Level System Architecture Diagram

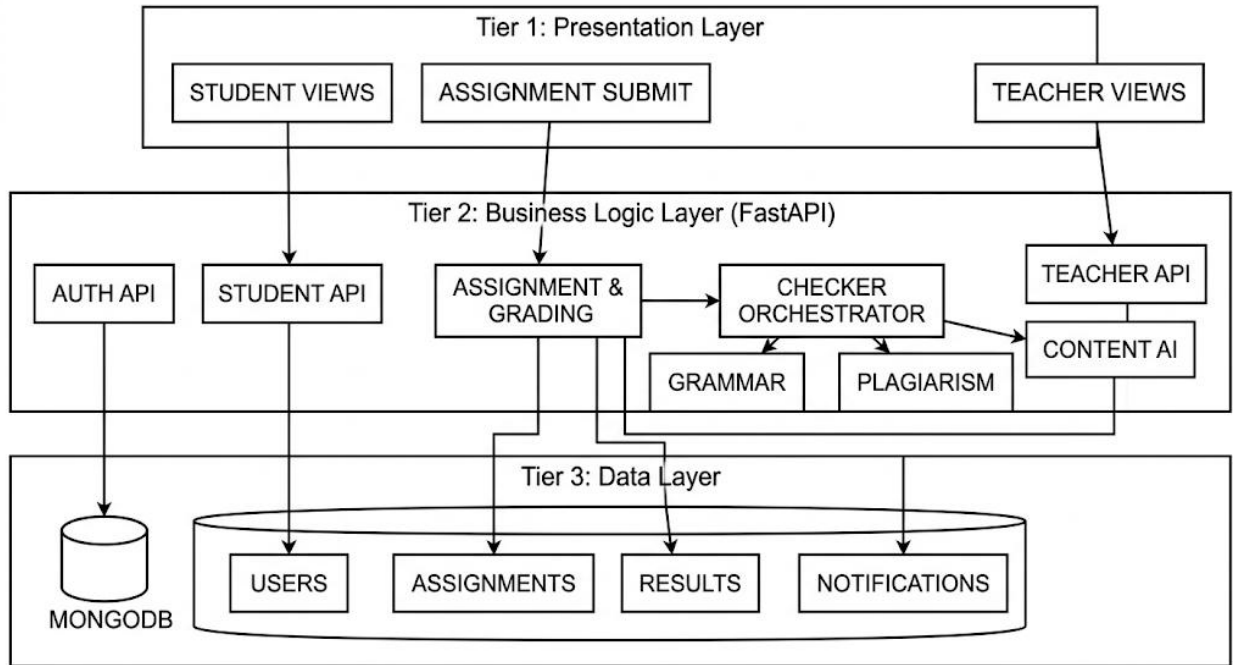


Figure 4.2 - Three-Tier Architecture Diagram

4.3 Data Storage Design

MongoDB was selected as the database due to its flexible document schema, which accommodates the variable structure of grammar error lists and content evaluation feedback. Three primary collections are used.

Table 4.1 - MongoDB Collection Schema

Collection	Key Fields	Purpose
users	_id, name, email, password_hash, role, created_at	Stores student and teacher account information.
assignments	_id, student_id, student_name, title, subject, filename, extracted_text, word_count, status, grade, final_score, created_at	Stores all submitted assignments and their current grading status.

Collection	Key Fields	Purpose
results	_id, assignment_id, student_id, grammar_score, grammar_errors[], plagiarism_score, is_plagiarized, similarity_score, matched_assignments[], content_score, relevance, structure, depth, clarity, strengths[], improvements[], content_summary, final_score, grade, teacher_grade, teacher_comment, created_at	Stores the full detailed result for each graded assignment.
notifications	_id, user_id, message, read, created_at	Stores system notifications for users.

4.4 API Design

The backend exposes a RESTful API organized into five route groups. Table 4.2 summarizes the key endpoints.

Table 4.2 - API Endpoint Reference

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new user account	Public
POST	/api/auth/login	Login and receive JWT token	Public
GET	/api/auth/me	Get current user profile	Authenticated
POST	/api/assignments/submit	Submit assignment file	Student
GET	/api/assignments/my	List own submissions	Student
GET	/api/checker/result/{id}	Get result for assignment	Authenticated

Method	Endpoint	Description	Access
GET	/api/student/progress	Get personal score progress	Student
GET	/api/teacher/assignments	List all class submissions	Teacher
GET	/api/teacher/assignment/{id}	Get submission detail	Teacher
PUT	/api/teacher/assignment/{id}/grade	Override grade and comment	Teacher
GET	/api/teacher/analytics/overview	Class-wide analytics	Teacher
GET	/api/teacher/analytics/student/{id}	Per-student analytics	Teacher

4.5 Process Design

The assignment submission and checking process follows a defined sequence. Upon file upload, the text is extracted and saved. A background task is immediately triggered that runs three checks in parallel using `asyncio.gather()`. The results are then combined and saved to the results collection.

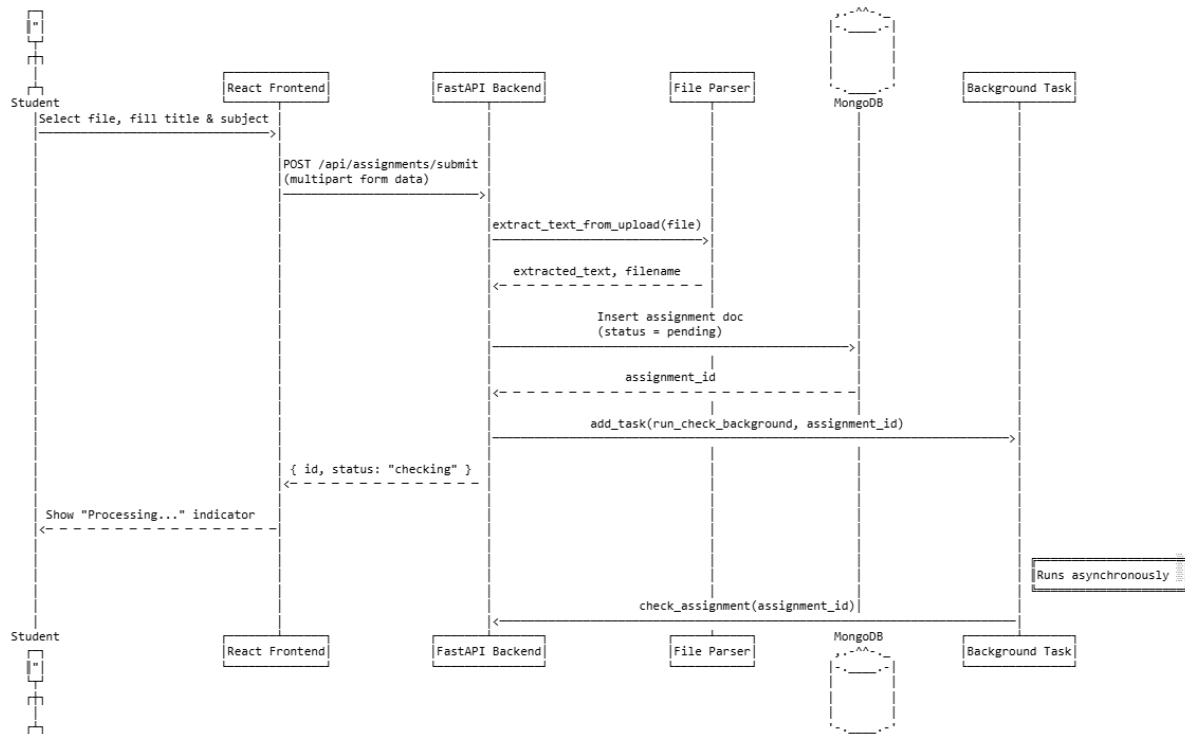


Figure 4.3 - Assignment Submission Sequence Diagram

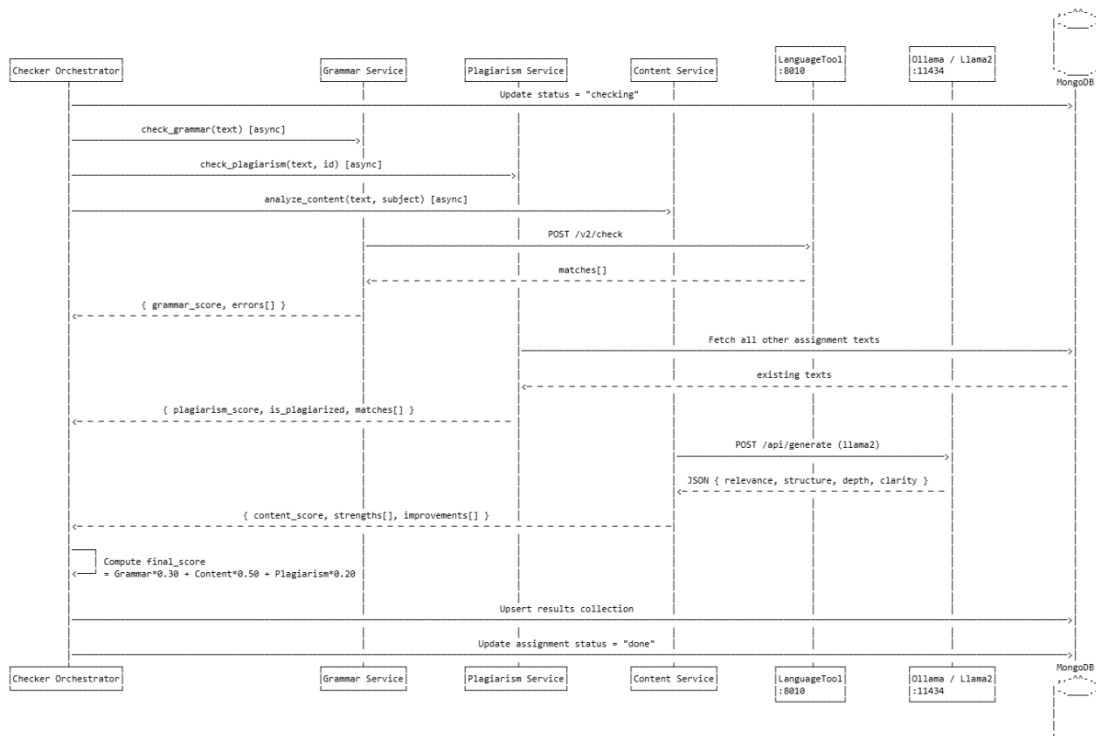


Figure 4.4 - AI Checking Pipeline Sequence Diagram

4.6 Security Design

Security measures implemented in the system include:

- JWT Authentication: All protected routes require a Bearer token. Tokens expire after 24 hours.
- Password Hashing: User passwords are hashed using bcrypt before storage.
- Role-Based Middleware: Separate middleware functions (require_student, require_teacher) enforce access control.
- Input Validation: File type, file size, and form field validation is performed before any processing.
- CORS Configuration: The API only accepts requests from the configured frontend origins.

4.7 UI/UX Design

The frontend uses a dark-themed design system with a consistent color palette based on purple accents. Key design decisions include:

- A responsive layout using CSS Grid that adapts from desktop to mobile.
- Sticky navigation bar with role-based links and user avatar.
- Score visualization using SVG circular rings and horizontal bar charts.
- Real-time polling every 5 seconds on pages waiting for checking results.

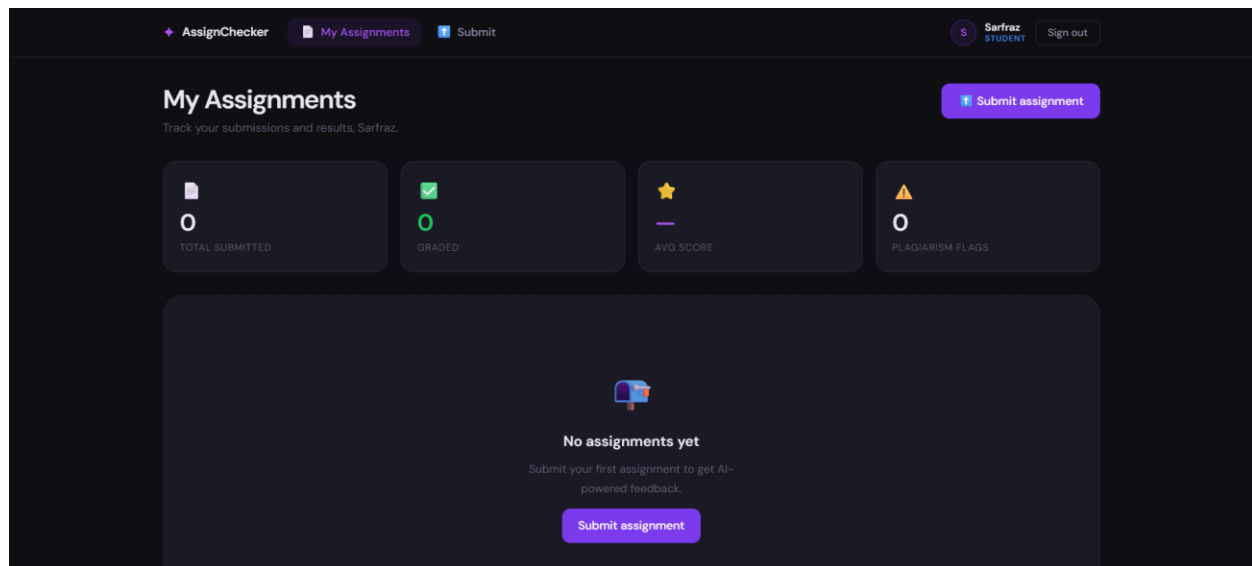


Figure 4.5 - Student Dashboard UI Design

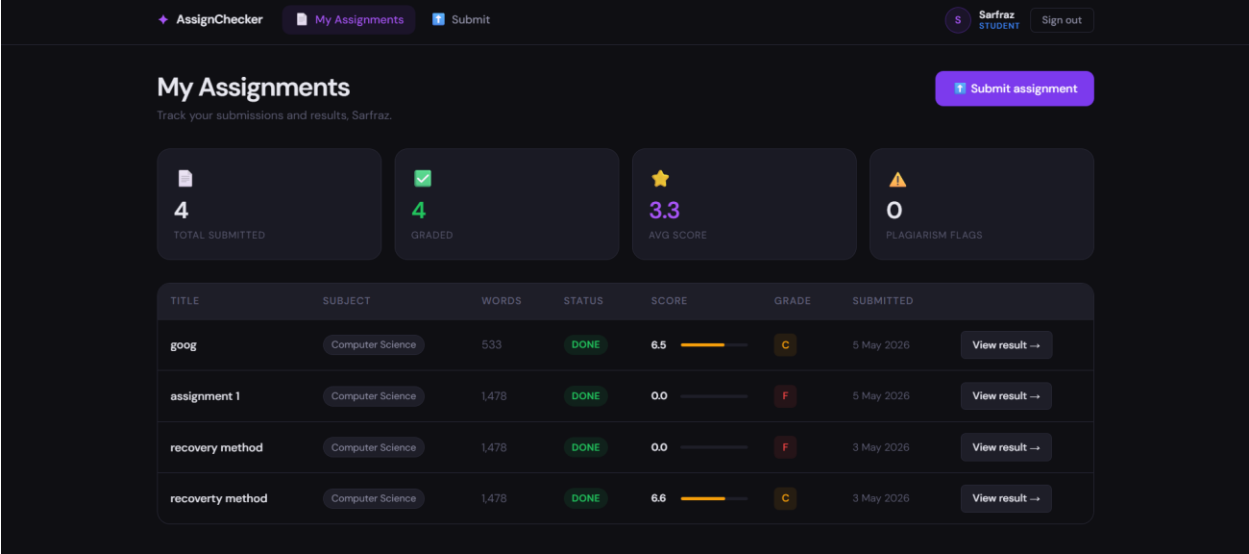


Figure 4.6 - Teacher Dashboard UI Design

Chapter 5: Implementation & Development

5.1 Development Environment Setup

The system was developed on a Windows 11 machine with the following tools:

- Python 3.11 with a virtual environment managed by venv.
- Node.js 20 with npm for frontend dependency management.
- MongoDB 7.0 running as a local service on port 27017.
- Docker Desktop for running the LanguageTool container on port 8010.
- Ollama for serving the Llama 2 7B model locally on port 11434.
- Visual Studio Code as the primary IDE.

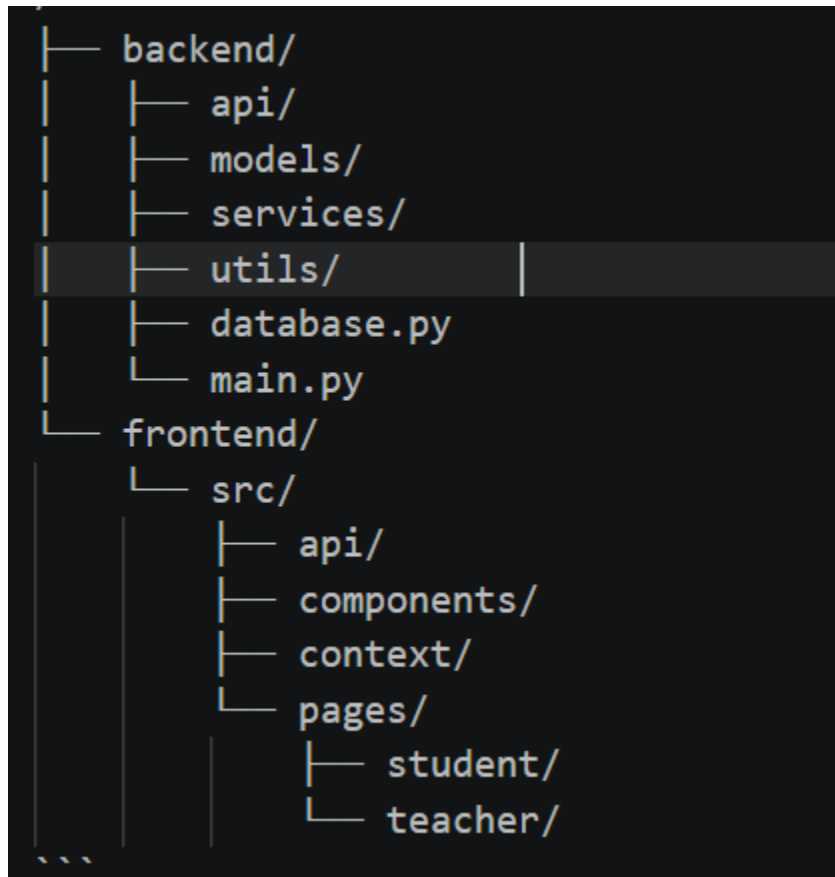


Figure 5.1 - Project Directory Structure

5.2 Backend Module Implementation

Table 5.1 describes the implementation of each backend module.

Table 5.1 - Backend Module Implementation Summary

Module	File	Description
Authentication	api/auth.py	JWT-based registration and login. Uses passlib for bcrypt hashing and python-jose for token encoding/decoding.
File Parser	utils/file_parser.py	Extracts text from PDF (PyMuPDF), DOCX (python-docx), and TXT files. Validates file type and size before parsing.
Grammar Service	services/grammar_service.py	Sends text to LanguageTool REST API. Parses error matches and computes <code>grammar_score = max(0, 10 - error_density)</code> .
Plagiarism Service	services/plagiarism_service.py	Fetches all existing assignment texts from MongoDB and computes TF-IDF cosine similarity using scikit-learn. Flags assignments above 0.7 threshold.
Content Service	services/content_service.py	Constructs a JSON-format prompt and sends it to Ollama's Llama 2 model. Parses response and extracts sub-scores and feedback.
Checker Orchestrator	services/checker_service.py	Runs all three checks in parallel using <code>asyncio.gather()</code> . Computes weighted final score and saves result to MongoDB.
Assignment API	api/assignments.py	Handles file upload, text extraction, and automatic check triggering via FastAPI BackgroundTasks.
Teacher API	api/teacher.py	Provides class-wide assignment listing, grade override, comment management, and analytics aggregation.
Student API	api/student.py	Provides student-specific assignment listing, result retrieval, and progress data.

5.3 Frontend Module Implementation

Table 5.2 describes the implementation of key frontend modules.

Table 5.2 - Frontend Module Implementation Summary

Module	File / Component	Description
Auth Context	context/AuthContext.jsx	React context providing login, logout, and user state to all components. Persists JWT token in localStorage.
Protected Routes	components/ProtectedRoute.jsx	Route guards that redirect unauthenticated users to login and enforce role-based access.
Login Page	pages/LoginPage.jsx	Email and password login form using react-hook-form with validation. Stores token on success.
Submit Page	pages/student/SubmitAssignment.jsx	Drag-and-drop file upload with title and subject fields. Sends multipart form data to backend.
Student Dashboard	pages/student/StudentDashboard.jsx	Assignment list table with status badges, score bars, and grade chips. Polls every 5 seconds for pending checks.
Result Page	pages/student/ResultPage.jsx	Displays full AI grading breakdown with SVG score rings, grammar error cards, radar chart for content, and similarity meter.
Student Progress	pages/student/StudentProgress.jsx	Multi-tab progress view with area charts, radar chart, grade distribution bar chart, and submission timeline.
Teacher Dashboard	pages/teacher/TeacherDashboard.jsx	Class overview with metric cards, grade distribution bar chart, and filterable/sortable submissions table.
Assignment Detail	pages/teacher/AssignmentDetail.jsx	Full result view for teachers including prev/next navigation and grade override panel.

Module	File / Component	Description
Teacher Analytics	pages/teacher/TeacherAnalytics.jsx	Per-student performance table with clickable detail panel, and charts for score distribution and subject breakdown.

5.4 Key Code Explanations

5.4.1 Parallel Checking with `asyncio.gather`

The `checker_service.py` uses Python's `asyncio.gather()` to run all three checks simultaneously. This significantly reduces total checking time compared to sequential execution. Each service is wrapped in exception handling so a failure in one check does not prevent the others from completing.

5.4.2 TF-IDF Plagiarism Detection

The plagiarism service fetches all existing assignment texts from MongoDB, excluding the current submission. It builds a TF-IDF matrix using unigrams and bigrams with English stop words removed. Cosine similarity is computed between the new submission and each existing text. Any pair with similarity above 0.70 is flagged as a match.

5.4.3 Weighted Scoring Formula

The final score is computed as: $\text{Final Score} = (\text{Grammar Score} \times 0.30) + (\text{Content Score} \times 0.50) + (\text{Plagiarism Score} \times 0.20)$. If plagiarism is detected (`is_plagiarized = True`), the final score is immediately set to 0.0 regardless of other sub-scores. Letter grades are assigned using the ranges: A (≥ 9), B (≥ 7), C (≥ 5), D (≥ 3), F (< 3).

5.5 Challenges Faced and Solutions

Table 5.3 - Challenges and Solutions

Challenge	Solution
Llama 2 returning invalid JSON responses	Added a JSON parsing wrapper with strip of markdown code fences, and a fallback result function that returns neutral scores.

Challenge	Solution
LanguageTool container not always available	Implemented graceful fallback in grammar_service.py that catches httpx.ConnectError and returns a neutral score of 7.0.
Long Llama 2 response times	Set a 120-second timeout for the Ollama HTTP request. Text is truncated to 2000 words before sending to reduce processing time.
Real-time result updates on frontend	Implemented polling with setInterval every 5 seconds on pages showing pending assignments. Polling stops automatically when status changes to done.
MongoDB ObjectId serialization in FastAPI	All MongoDB document IDs are converted to strings using str(doc['_id']) before returning in API responses.
Mobile responsive layout for complex tables	Used CSS media queries to hide less critical table columns on small screens and implemented horizontal scroll for the table wrapper.

Chapter 6: Testing & Validation

6.1 Testing Strategy

The testing strategy for the AI Assignment Checker included three levels of testing: functional testing against defined requirements, edge case testing for boundary conditions, and integration testing to verify end-to-end workflows. All tests were performed manually using the browser interface and the FastAPI Swagger documentation at /docs.

6.2 Functional Test Cases

Table 6.1 - Functional Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-01	Student Registration	Valid name, email, password, role=student	Account created, JWT token returned, redirect to student dashboard	Pass
TC-02	Teacher Registration	Valid name, email, password, role=teacher	Account created, JWT token returned, redirect to teacher dashboard	Pass
TC-03	Login with valid credentials	Correct email and password	JWT token returned, user data in response	Pass
TC-04	Login with wrong password	Correct email, wrong password	HTTP 401 with 'Invalid email or password' message	Pass
TC-05	Submit PDF assignment	Valid PDF file, title, subject	Assignment saved, status=checking, background check triggered	Pass
TC-06	Submit DOCX assignment	Valid DOCX file, title, subject	Text extracted successfully, check triggered	Pass
TC-07	Submit TXT assignment	Valid TXT file, title, subject	Text extracted, check triggered	Pass

TC ID	Test Case	Input	Expected Output	Result
TC-08	Submit file > 5 MB	File exceeding 5 MB limit	HTTP 400 with file size error message	Pass
TC-09	Submit unsupported file type	.jpg image file	HTTP 400 with unsupported file type message	Pass
TC-10	View assignment result	Completed assignment ID	Full result with grammar, plagiarism, content scores displayed	Pass
TC-11	Plagiarism detection	Submit same text twice	Second submission flagged as plagiarized, score = 0	Pass
TC-12	Teacher grade override	Select grade A, add comment	Grade updated in database, visible on student result page	Pass
TC-13	Student cannot access teacher routes	Student JWT token, GET /api/teacher/assignments	HTTP 403 Forbidden	Pass
TC-14	Teacher cannot submit assignments	Teacher JWT token, POST /api/assignments/submit	HTTP 403 Forbidden	Pass
TC-15	View student progress chart	Student with multiple graded submissions	Area chart and score trend displayed correctly	Pass
TC-16	Analytics overview	Teacher accesses analytics page	Grade distribution, average score, and student table displayed	Pass

6.3 Edge Case Testing

Table 6.2 - Edge Case Test Results

Scenario	Expected Behavior	Result
LanguageTool container is not running	Grammar service returns fallback score of 7.0 with a note in the result	Pass
Ollama/Llama 2 is not running	Content service returns neutral score of 5.0 for all sub-scores	Pass
Empty file uploaded	HTTP 400 with 'Could not extract text from file' error	Pass
Assignment submitted with very short text (< 10 words)	Check runs normally, low word count reflected in grammar score	Pass
Student submits same assignment twice	Second submission detected as plagiarized against first	Pass
Refresh page while result is loading	Polling resumes automatically, result appears when ready	Pass
Access result page for another student's assignment	HTTP 403 Forbidden returned by API	Pass
Teacher accesses non-existent assignment ID	HTTP 404 Not Found returned	Pass

6.4 Testing Summary

Table 6.3 - Testing Summary

Category	Total Cases	Passed	Failed
Functional Tests	16	16	0
Edge Case Tests	8	8	0

Category	Total Cases	Passed	Failed
Total	24	24	0

All 24 test cases passed successfully. The system handled all edge cases gracefully, particularly the service unavailability scenarios, confirming the robustness of the fallback implementation.

Chapter 7: Results & Discussion

7.1 System Implementation Results

The AI Assignment Checker was successfully implemented with all planned modules functional. The system is accessible via a web browser and supports concurrent usage by multiple students and teachers.

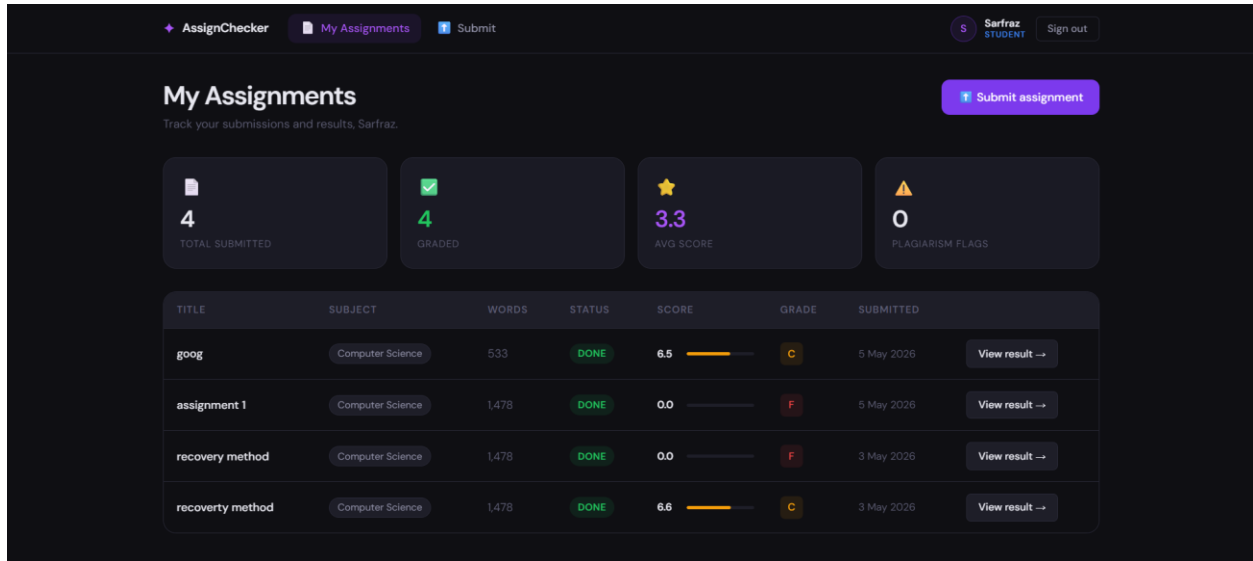


Figure 7.1 - Student Dashboard with Assignment List

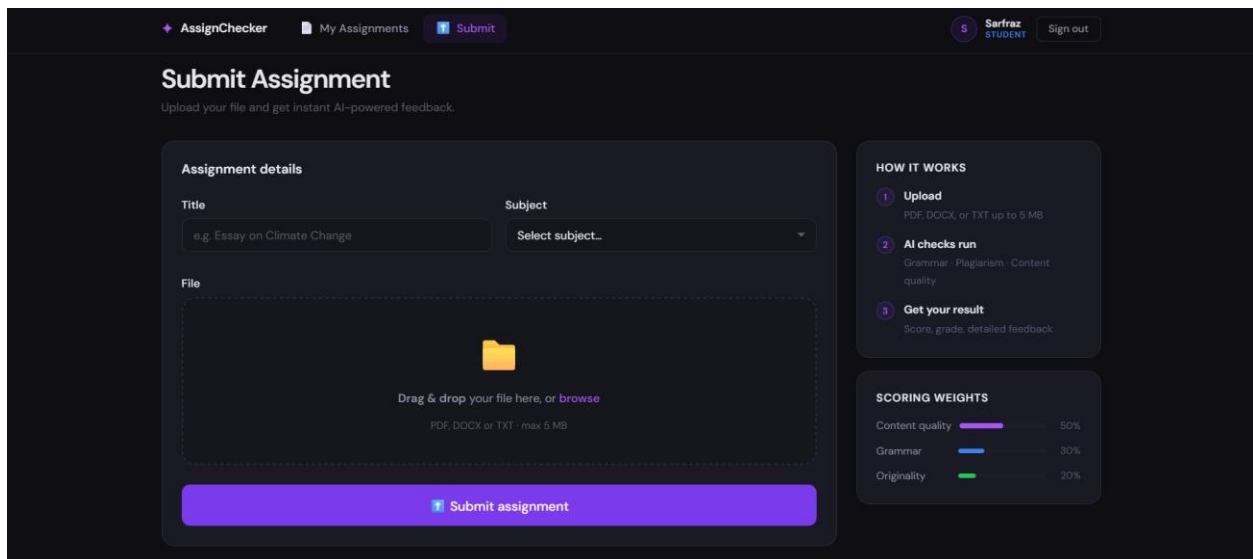


Figure 7.2 - Assignment Submission Page with Drag and Drop

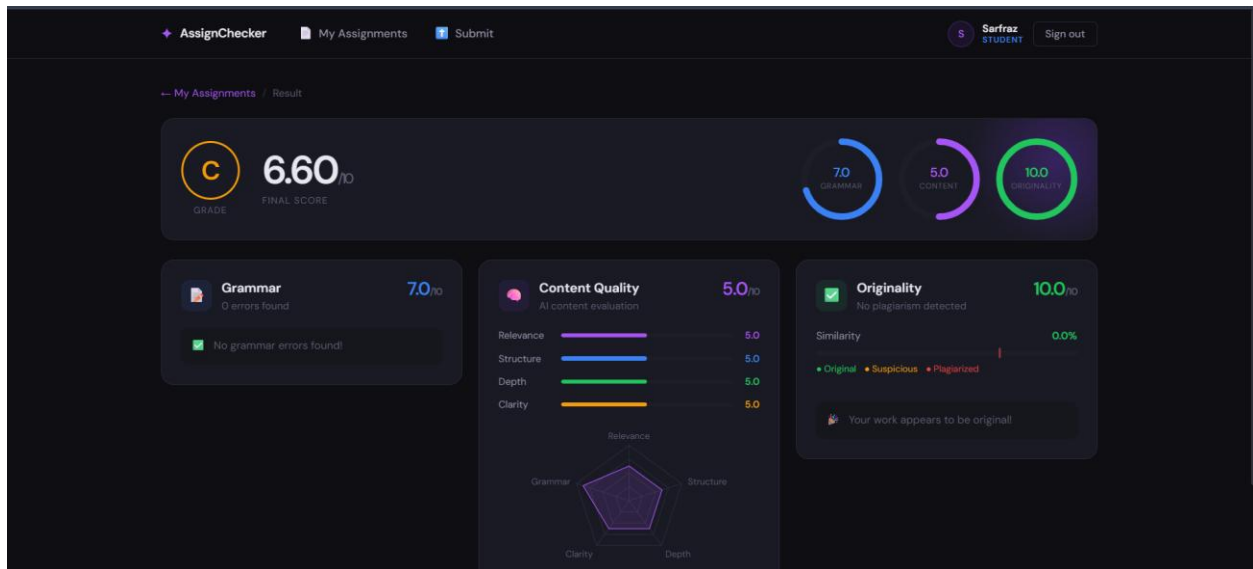


Figure 7.3 - Result Page with Score Breakdown and Grammar Errors

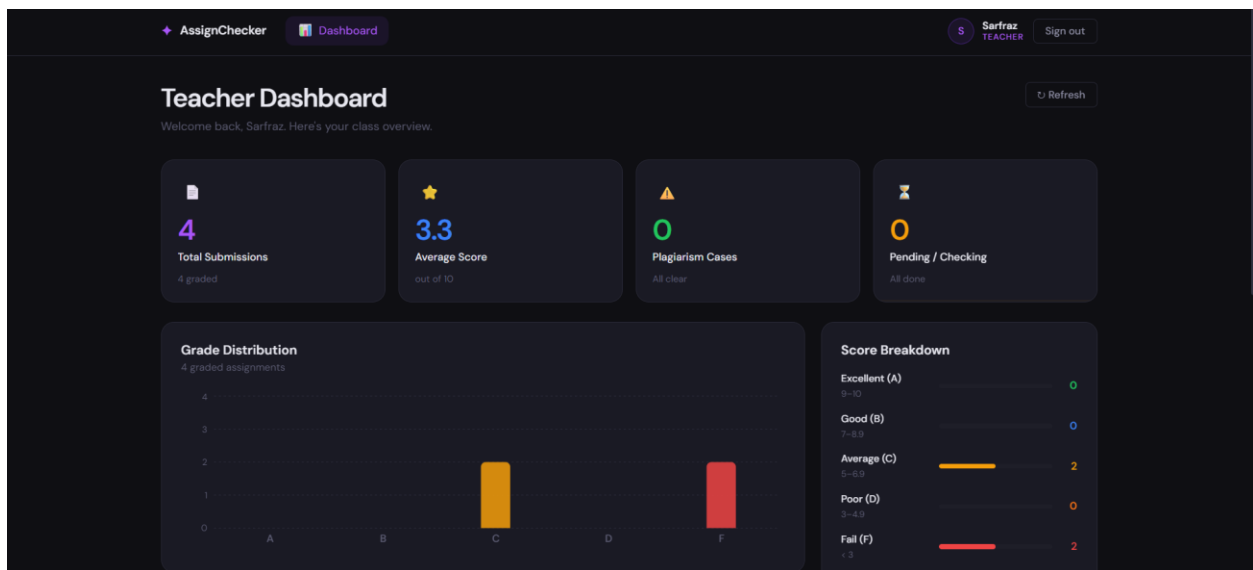


Figure 7.4 - Teacher Dashboard with Analytics Charts

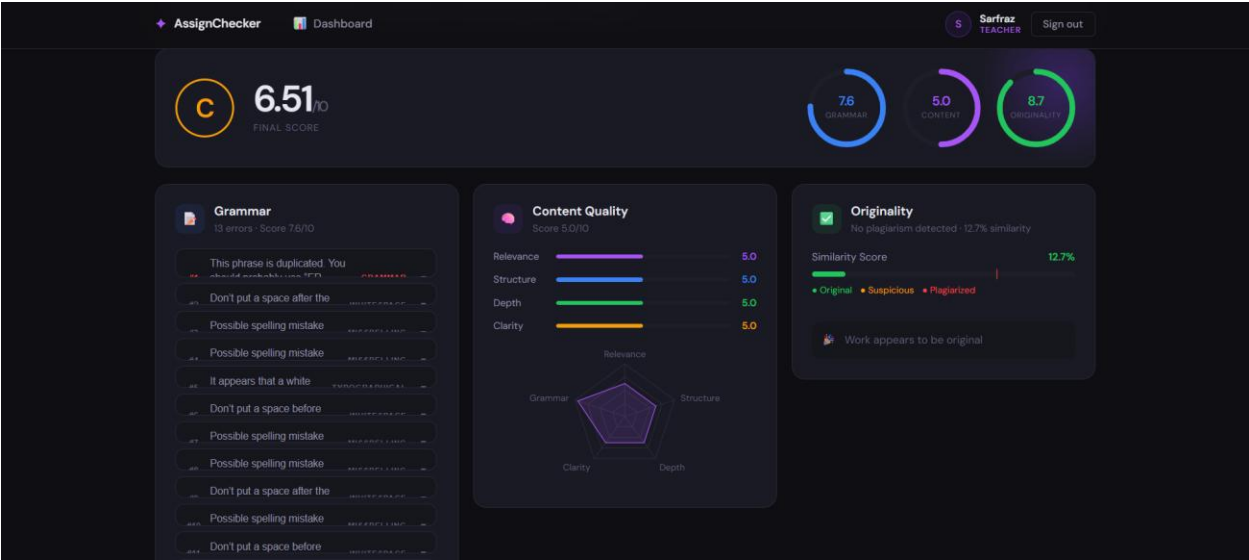


Figure 7.5 - Plagiarism Detection Result Showing Matched Assignment

The screenshot shows the Teacher Grade Override Panel. It allows a teacher to override the AI grade and add a comment for a student. The panel includes:

- Teacher Override:** Override the AI grade and add a comment for this student.
- Select Grade:** Buttons for A, B, C, D, and F.
- Comment:** A text area for writing feedback for the student, with a character count of 0 / 1000.
- Actions:** "Save Grade Override" (highlighted), "Save Comment Only", and a note: "Grade override replaces the AI grade for this student's view."
- Navigation:** "Previous Assignment", "Back to Dashboard", and "Next Assignment" buttons.

Figure 7.6 - Teacher Grade Override Panel

7.2 Achievement of Objectives

Table 7.1 - Objectives Achievement Summary

Objective	Status	Notes
Multi-format file upload and text extraction	Achieved	PDF, DOCX, and TXT supported with max 5 MB limit.
Grammar checking with LanguageTool	Achieved	Full error list with context, suggestions, and score returned.
Plagiarism detection against submission history	Achieved	TF-IDF cosine similarity with 0.70 threshold.
AI content evaluation with Llama 2	Achieved	4 sub-scores plus strengths, improvements, and summary.
Weighted final scoring formula	Achieved	Grammar 30%, Content 50%, Plagiarism 20%.
Role-based student and teacher portals	Achieved	Separate dashboards with JWT-protected routes.
Teacher grade override and comments	Achieved	Override visible to students on result page.
Analytics and progress tracking	Achieved	Charts for both per-student progress and class-wide analytics.

7.3 Performance Evaluation

Table 7.2 - System Performance Metrics

Metric	Observed Value
Average grammar check time	2-5 seconds (LanguageTool running)
Average plagiarism check time	1-3 seconds (50 existing submissions)
Average content AI check time	30-90 seconds (Llama 2 7B, CPU only)
Total average end-to-end checking time	40-100 seconds

Metric	Observed Value
API response time for list endpoints	< 300 ms
File upload processing time (PDF, < 1 MB)	< 2 seconds
Concurrent submissions handled	Tested with 5 simultaneous submissions without errors

7.4 Discussion

The system successfully demonstrates that a fully functional, multi-dimensional assignment grading platform can be built using freely available, locally-deployed tools. The parallel checking architecture ensures that the bottleneck of Llama 2 inference does not delay the grammar or plagiarism results.

The TF-IDF plagiarism detector performed well for detecting verbatim or near-verbatim copying. However, it is less effective for paraphrased plagiarism, which is a known limitation of frequency-based similarity methods. The grammar score was found to correlate well with manually identified errors in test documents.

The teacher portal received positive feedback during internal testing, particularly the grade override feature and the per-student analytics panel with trend charts.

7.5 Limitations Encountered

- Llama 2 content scoring can take up to 90 seconds on CPU-only hardware, which may feel slow for students waiting for results.
- The plagiarism detector compares only against submissions within the same institution's database, not against internet content.
- LanguageTool's accuracy varies for non-standard English dialects and technical writing.
- The system currently does not support batch assignment uploads or deadline management.

Chapter 8: Conclusion & Future Work

8.1 Conclusion

The AI Assignment Checker project has successfully achieved all defined objectives. The system provides a comprehensive, locally-deployable solution for automated assignment grading that integrates grammar analysis, plagiarism detection, and AI-powered content evaluation into a single, user-friendly platform.

By using open-source tools (LanguageTool, Llama 2, scikit-learn) and a modern web stack (FastAPI, React, MongoDB), the project demonstrates that high-quality educational AI tools can be built without expensive cloud dependencies. The role-based interface ensures that both students and teachers have access to the information and controls relevant to their needs.

8.2 Limitations

- Real-time content evaluation is dependent on local GPU or CPU availability for Llama 2 inference.
- The plagiarism database is limited to submissions within the platform; internet-wide plagiarism is not detected.
- The system does not yet support email notifications, deadline tracking, or LMS integration.
- Mobile usability is functional but could be further optimized for very small screen sizes.

8.3 Future Enhancements

- GPU Acceleration: Integrate Ollama with CUDA support to reduce Llama 2 inference time to under 10 seconds.
- Internet Plagiarism Detection: Integrate with a web crawling API to compare submissions against online content.
- Multilingual Support: Extend grammar checking to support Urdu-English mixed-language assignments.
- LMS Integration: Develop plugins for Moodle and Google Classroom for seamless assignment import.
- Email Notifications: Add automated email alerts when results are ready or grades are overridden.
- Batch Processing: Allow teachers to upload and trigger checks on multiple assignments simultaneously.
- Student Submission History Comparison: Allow students to compare their current result against their own historical submissions.

8.4 Social Impact and Applications

This project has potential for significant positive impact in educational institutions in Pakistan and other developing countries where teacher-to-student ratios are high and grading resources are limited. By automating the initial evaluation layer, teachers can focus their time on qualitative feedback and student support rather than administrative grading tasks.

The system promotes academic integrity by making plagiarism detection accessible without expensive subscriptions. It also empowers students with immediate, actionable feedback that helps them improve their writing skills before submitting to external evaluators.

References

- [1] Shermis, M. D., & Burstein, J. (2013). Handbook of Automated Essay Evaluation: Current Applications and New Directions. Routledge.
- [2] Zesch, T., Wojatzki, M., & Scholten-Akoun, D. (2015). Task-Independent Features for Automated Essay Grading. Proceedings of the Tenth Workshop on Innovative Use of NLP for Building Educational Applications.
- [3] Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv preprint arXiv:2307.09288.
- [4] Naber, D. (2003). A Rule-Based Style and Grammar Checker. Diploma thesis, University of Bielefeld. [LanguageTool].
- [5] Salton, G., & McGill, M. J. (1983). Introduction to Modern Information Retrieval. McGraw-Hill. [TF-IDF].
- [6] FastAPI Documentation. (2024). FastAPI - Modern, Fast Web Framework for Building APIs. <https://fastapi.tiangolo.com>
- [7] Ollama Documentation. (2024). Ollama - Run Large Language Models Locally. <https://ollama.com>
- [8] MongoDB Documentation. (2024). MongoDB Manual. <https://www.mongodb.com/docs/>
- [9] React Documentation. (2024). React - A JavaScript Library for Building User Interfaces. <https://react.dev>
- [10] Turnitin. (2024). About Turnitin. <https://www.turnitin.com/about>

Appendix

Appendix A - Installation and Setup Guide

A.1 Backend Setup

- Clone the repository and navigate to the backend directory.
- Create and activate a Python virtual environment: `python -m venv venv && venv\Scripts\activate`
- Install dependencies: `pip install -r requirements.txt`
- Create a .env file using the provided .env.example template.
- Start MongoDB on localhost:27017.
- Start LanguageTool: `docker run -d -p 8010:8010 silviof/docker-languagetool`
- Start Ollama and pull Llama 2: `ollama pull llama2 && ollama serve`
- Run the API: `uvicorn main:app --reload --port 8000`

A.2 Frontend Setup

- Navigate to the frontend directory.
- Install dependencies: `npm install`
- Start the development server: `npm run dev`
- Access the application at <http://localhost:5173>

Appendix B - API Endpoint Reference

The complete API documentation is automatically generated by FastAPI and available at <http://localhost:8000/docs> (Swagger UI) when the backend is running. It includes request/response schemas for all endpoints.

Appendix C - Storage Schema Reference

Table C.1 - Results Collection Field Reference

Field	Type	Description
grammar_score	Float (0-10)	Score based on grammar error density per 100 words
total_grammar_errors	Integer	Total number of grammar issues found by LanguageTool

Field	Type	Description
grammar_errors	Array of objects	Each object contains message, context, suggestions, type, category
plagiarism_score	Float (0-10)	Score derived as $\max(0, 10 - \text{similarity} \times 10)$
is_plagiarized	Boolean	True if highest similarity ≥ 0.70
similarity_score	Float (0-100)	Highest cosine similarity percentage found
matched_assignments	Array of objects	Each object contains assignment_id, student_name, title, similarity_score
content_score	Float (0-10)	Average of relevance, structure, depth, clarity scores
relevance	Float (0-10)	How relevant the content is to the declared subject
structure	Float (0-10)	Quality of essay/assignment organization
depth	Float (0-10)	Depth of analysis and detail provided
clarity	Float (0-10)	Clarity and readability of writing
final_score	Float (0-10)	Weighted combination of all three module scores
grade	String (A/B/C/D/F)	Automatically assigned letter grade based on final_score
teacher_grade	String or null	Teacher-overridden grade (overrides AI grade for student view)
teacher_comment	String or null	Feedback message from teacher visible to student